Maintainer Prioritization in Registry-Scale Supply Chains: Multi-Objective Trade-offs, Representation Sensitivity, and Authority-Aware Hardening

Ashikul Islam, *Graduate Student Member, IEEE,* and Shahriar Hasan, *Member, IEEE*

Abstract—Centralized package registries such as npm rely on maintainer accounts as root authorities for software publication. However, prioritization algorithms for proactive supply-chain interventions (e.g., hardware security key distribution, multi-factor authentication enforcement, and manual audit campaigns) face fundamental mathematical and structural dilemmas. In this paper, we present an empirical study of maintainer prioritization across 25,228 npm packages, 12,217 maintainer accounts, and 7,171 continuous-integration workflow repositories.

We formalize maintainer selection as an optimization problem across three competing objectives: submodular coverage $C(S)$, modular scenario exposure reduction $H(S)$, and supermodular all-source exclusion $P(S)$. We prove that greedy coverage and exact-union ranking optimize diametrically opposing properties: at a budget of 100 identities (0.819% of maintainers), greedy coverage covers 5,538 repositories (77.23%) but leaves 85.55% of downstream endpoints vulnerable to unselected sources ($P = 800$), whereas exact union eliminates 33.79% of exposure mass but covers 1,131 fewer repositories ($\Delta C = -1,131$). To resolve this tension, we formulate a monotone submodular-modular hybrid scalarization $J_\alpha(S)$. Across both package-projected and version-continuous graphs, the Pareto frontier reveals an empirical sweet spot at $\alpha = 0.50$, capturing over 92% of maximum exposure reduction ($H = 31.19\%$ on name graph; $H = 38.65\%$ on version graph) while maintaining broad 70% ecosystem coverage ($C \geq 5,000$). Furthermore, direct optimization of supermodular exclusion via a greedy minimum-remaining-source heuristic breaks past 1,000 repositories, securing $P = 1,022$ fully defended endpoints (+27.8% over greedy; +91.4% over exact union).

Investigating heterogeneous maintainer risk priors reveals that account-level metrics substantially misspecify fine-grained risk: unweighted strategies capture only 23.40%–27.32% of package-level sole-maintainer exposure mass ($H_{\text{pkg-sole}}$), whereas weight-aware package-level optimization captures 67.37% (+40.05 pp). Finally, through a retrospective audit of 45 pre-compromise snapshots and 1,848 reconstructed SemVer edges, we show that 97.8% of incidents exhibit predecessor version availability and 93.3% exhibit repository descriptors suitable for automated gating, while official cryptographic provenance attestations remain absent across historical metadata. Our findings provide formal principles and operational tooling for registry-level and enterprise supply-chain defenses.

Index Terms—Software supply chain security, npm ecosystem, maintainer prioritization, multi-objective optimization, submodularity, dependency graphs, release authority.

1 Introduction

Modern software engineering is inextricably bound to open-source package ecosystems. Centralized package managers, led by npm with over 3 million registered packages and billions of weekly downloads, enable rapid assembly of complex applications through deep, transitive dependency graphs [1]. However, this structural efficiency creates an asymmetric attack surface: an adversary who successfully compromises a single maintainer account can execute malicious code across tens of thousands of downstream continuous-integration (CI) pipelines, server environments, and end-user systems [2], [3].

Securing these registry-scale ecosystems requires defensible resource allocation. Ecosystem guardians (such as npm, GitHub, and the Open Source Security Foundation [OpenSSF]) operate under severe capacity constraints: manual source-code audits, hardware security token distribution, mandatory multi-factor authentication (MFA) rollouts, and privileged-access review programs cannot be extended to all 12,000+ active maintainers simultaneously [4], [5]. Consequently, defenders rely on algorithmic maintainer prioritization to identify which accounts should receive prioritized controls.

Despite the critical nature of this task, prior prioritization heuristics have operated under unverified assumptions. Commonly deployed heuristics—such as ranking maintainers by total portfolio size, raw download numbers, or greedy unconstrained downstream coverage—treat software dependency graphs as simple reachability networks [6], [7]. In doing so, existing approaches conflate three fundamentally distinct defensive objectives:

• *A. Islam and S. Hasan are with the Department of Computer Science and Engineering, Hajee Mohammad Danesh Science and Technology University (HSTU), Dinajpur 5200, Bangladesh.*
E-mails: ashik@hstu.ac.bd, shahriar@hstu.ac.bd.

Manuscript submitted to IEEE Transactions on Software Engineering (TSE), October 2026.

1) **Broad Ecosystem Coverage ($C(S)$):** The total number of distinct downstream client repositories that depend on at least one package authored by the selected maintainers. This is a classic submodular set function [8].
2) **Scenario Exposure Reduction ($H(S)$):** The fraction of maintainer-repository dependency pairs eliminated under a probabilistic model of independent account compromise. Because each account compromise represents a distinct historical event, this is a modular (additive linear) set function.
3) **All-Source Exclusion ($P(S)$):** The number of downstream client repositories whose complete set of upstream maintainers is fully contained within the hardened cohort, rendering the client impervious to single-maintainer account takeovers. Because protection requires all upstream maintainers to be selected, this is a supermodular set function.

In this paper, we demonstrate that optimizing for broad coverage (C) actively undermines exposure reduction (H) and full protection (P), creating severe residual exposure across the ecosystem. When defenders maximize greedy coverage, maintainer multi-homing and co-maintenance overlap induce extensive masking: accounts managing ubiquitous shared libraries (e.g., maintainer `qix` managing `debug`) are discarded by greedy coverage heuristics because their downstream endpoints are already partially touched by other maintainers, leaving thousands of repositories residually exposed.

Furthermore, we show that standard dependency graph representations introduce massive measurement distortions. Coarse package-name projections—which collapse all historical versions of a package into a single node—inflate downstream attack surfaces by 42.44% (over 196,000 phantom pairs) and distort individual package blast radii by up to 287%, misallocating defensive investments.

To address these challenges, this study presents a comprehensive, multi-objective theoretical and empirical investigation of maintainer prioritization across 25,228 npm packages, 12,217 maintainer accounts, and 7,171 workflow repositories. Specifically, we investigate five core Research Questions:

- **RQ1 (Objective Trade-offs):** How do distinct coverage $C(S)$ and scenario exposure reduction $H(S)$ diverge under operational budgets, and what is the magnitude of the descriptive trade-off?
- **RQ2 (Residual Exposure & Supermodular Protection):** What fraction of covered repositories retain unhardened upstream attack paths under standard heuristics, and can dedicated supermodular optimization maximize all-source exclusion $P(S)$?
- **RQ3 (Representation Sensitivity):** How does version continuity alter downstream reach closures, maintainer rankings, and cross-representation selection penalties compared to name-collapsed graphs?
- **RQ4 (Multi-Objective Optimization & Heterogeneous Risk):** Can a hybrid scalarization $J_\alpha(S)$ achieve a Pareto-optimal frontier between coverage and exposure, and how do heterogeneous maintainer risk models (solo-maintainer bottlenecks, log-portfolio scaling, package-level reach) reshape prioritization?
- **RQ5 (Temporal Reconstruction & Release Authority):** How effectively can time-bounded SemVer dependency resolution eliminate future-leakage bias, and what pre-event metadata discontinuities characterize real-world compromise snapshots?

1.1 Contributions

The primary contributions of this work are as follows:

1) **Multi-Objective Optimization Framework:** We formalize maintainer prioritization across submodular (C), modular (H), and supermodular (P) objectives, proving that single-objective baselines induce extreme polarization.
2) **The $\alpha = 0.50$ Pareto Sweet Spot:** We formulate a monotone submodular-modular scalarization $J_\alpha(S)$ that captures $> 92\%$ of maximum exposure reduction while maintaining 70% broad repository coverage.
3) **Supermodular Exclusion Breakthrough:** We design a greedy minimum-remaining-source heuristic that breaks past 1,000 fully protected repositories at budget 100 ($P = 1,022$, $+27.8\%$ over greedy, $+91.4\%$ over exact union).
4) **Package-Level Sole-Maintainer Risk Theorem:** We prove that account-level metrics misspecify vulnerability bottlenecks: weight-aware package-level optimization captures 67.37% of solo-maintainer exposure mass (+40.05 pp over unweighted union).
5) **Historical SemVer & Release-Authority Audit:** We evaluate 45 pre-compromise snapshots with 1,848 reconstructed SemVer edges, quantifying that 97.8% of incidents exhibit predecessor availability suitable for automated release gating.

2 Related Work

Our research intersects five major areas of empirical software engineering and software supply-chain security: structural ecosystem measurement, malicious package detection, continuous-integration workflow analysis, registry governance and provenance, and temporal dependency reconstruction.

2.1 Ecosystem Structure and Centrality-Based Prioritization

Pioneering empirical studies established that open-source package registries exhibit extreme structural asymmetry and small-world properties. Zimmermann et al. [1] analyzed npm's dependency topology, revealing that a tiny core of central packages and maintainers transitively affects a vast majority of the ecosystem. Zahan et al. [9] and Liu et al. [7] demonstrated that vulnerabilities propagate rapidly through deep, hierarchical trees, identifying "weak links" in software supply chains.

Building upon structural topology, recent work has proposed maintainer and package prioritization mechanisms. Wang [6] explored centrality-based protection strategies

within dependency networks. However, as we demonstrate in this paper, classical network centrality metrics (e.g., degree centrality, PageRank, or raw portfolio size) fail to account for the submodular-modular dichotomy between distinct client coverage and scenario exposure reduction, nor do they account for co-maintenance overlap in shared critical libraries.

2.2 Malicious Package Benchmarks and Detection Limits

A parallel line of security research focuses on detecting malicious packages published via typosquatting, dependency confusion, and account takeovers [2], [10]. Benchmark evaluations, such as Guo et al.'s extensive study of npm malware detectors [11], show that static analysis tools face severe limitations due to obfuscation, dynamic runtime execution, and environment-keyed payloads.

Because reactive post-publication malware detection suffers from non-trivial evasion rates, proactive hardening of legitimate, high-leverage maintainer accounts remains the primary line of registry defense [4], [5]. Our work complements malware detection by optimizing how defenders allocate proactive security interventions before credentials or tokens are compromised.

2.3 Continuous-Integration Workflows and Reachability Limits

Software supply-chain risks manifest concretely in downstream CI/CD pipelines. Koishybayev et al. [3] and Muralee et al. [12] demonstrated that GitHub Actions workflows frequently execute third-party build scripts with access to repository secrets, cloud deployment credentials, and package publish tokens.

Crucially, recent empirical studies have emphasized that package-level dependency relationships do not equal actual code execution. Weeraddana, Alfadel, and McIntosh [13] established that a substantial fraction of dependencies declared in npm projects represent "dependency-induced waste"—packages invoked during development but never executed in production or continuous integration. Furthermore, an extensive benchmark by Gao et al. [14] proved that 99% of client vulnerabilities originate in transitive dependencies, yet **81.26% of vulnerabilities flagged by package-level reachability are false positives at the function call-graph level**. Jayasuriya et al. [15] observed similar granularity effects when analyzing syntactic breaking changes across versions. In our work, we address this fundamental reachability gap by evaluating both coarse package-name projections and version-continuous graphs, explicitly qualifying topological reachability as an upper-bound potential review surface.

2.4 Registry Governance, Cryptographic Provenance, and Signing

In response to account compromises, registries and standards bodies have introduced cryptographic signing and provenance attestations. Schorlemmer et al. [4] audited signing across npm, PyPI, Maven, and RubyGems, finding that adoption remains minimal and unevenly distributed. Okafor, Torres-Arias, and Davis [5] interviewed industry practitioners, revealing significant operational friction in managing cryptographic keys and configuring automated pipeline attestations.

Simultaneously, He, Vasilescu, and Kästner [16] demonstrated that local dependency version pinning is insufficient to protect clients against upstream attacks, as adversaries exploit unpinned sub-dependencies and registry-level re-resolutions. These findings emphasize that registry-level maintainer account controls are indispensable.

2.5 Temporal Reconstruction and Empirical Standards

A critical methodological vulnerability in supply-chain studies is future-leakage bias—evaluating historical attacks using contemporary registry graphs. Pinckney et al. [17] introduced `npm-follower` to mirror registry event streams, while Rahman et al. [18] released `CHRONO-RESOLUTION` to reconstruct exact dependency trees at release points. Shafin and Ahmed [19] highlighted the necessity of time-indexed forecasting.

Methodologically, our evaluation adheres strictly to the ACM SIGSOFT Empirical Standards [20] and Kitchenham et al.'s updated guidelines on empirical software engineering [21]. By avoiding pseudo-replication—specifically rejecting the treatment of fixed-graph nested budgets or repeated random orders as independent sampling units—our study provides rigorous, reproducible descriptive and Pareto-optimal findings.

3 Data Corpus and Graph Construction

To conduct our empirical investigations, we construct and analyze two distinct mathematical graph representations derived from a centralized snapshot of the npm ecosystem, enriched with continuous-integration metadata and repository links from deps.dev [22] and GitHub.

3.1 Study Universe and Empirical Dimensions

The empirical corpus analyzed in this study comprises:

- **Packages (P):** 25,228 unique npm package names.
- **Package Versions (V):** 22,340 versions possessing detailed dependency manifests.
- **Maintainer Accounts (M):** 12,217 unique publisher accounts, spanning 79,405 author-package assignments.
- **Dependency Edges ($E_{\mathbf{dep}}$):** 88,113 unique directed package-to-package dependency relationships (48,705 direct dependencies, 73,180 transitive relations).
- **Workflow-Verified Repositories ($R_{\mathbf{wf}}$):** 7,171 distinct client software repositories with verified, active GitHub Actions continuous-integration workflows.

3.2 Two Graph Representations: Name-Projected vs. Version-Continuous

A central contribution of our study is evaluating prioritization sensitivity across two structural graph models:

1) **Package-Projected Graph ($G_{\mathbf{proj}}$):** Standard in the literature, this representation collapses all published

TABLE 1: Systematic positioning of our Q1 empirical investigation relative to major related literature.

Research Dimension	Structural Topology & Centrality	Malicious Package Detectors	CI/CD & Function Call-Graphs	This Study (Q1 Journal Expansion)
Focus Literature	Zimmermann [1], Zahan [9], Wang [6]	Ohm [2], Sejfia [10], Guo [11]	Koishybayev [3], Weeraddana [13], Gao [14]	Formal multi-objective optimization (C, H, P, J_α), version continuity, release authority
Prioritization Metric	Degree centrality, PageRank, raw portfolio size	Reactive post-event incident signatures	Taint analysis on workflows, call-graph reachability	Submodular coverage, modular exposure reduction, supermodular all-source exclusion
Graph Representation	Coarse package-name projected graphs	Isolated package tarballs	Dynamic execution traces, static ASTs	Dual evaluation: Package-projected vs. Version-continuous closures (42.4% pair diff)
Maintainer Risk Prior	Uniform equal weight	Unweighted	Unweighted	Heterogeneous weights: Solo bottlenecks (H_{sole}), log-portfolio ($H_{\log}$), package-level sole reach ($H_{\text{pkg-sole}}$)
Temporal Grounding	Static single snapshot	Retrospective positive-only datasets	Static GitHub Actions scripts	45 historical pre-incident snapshots, 1,848 SemVer edges ($t_{\text{cutoff}} < t_{\text{incident}}$), 90-day prospective cohort

versions of package p into a single node. A directed edge $(u, v) \in E_{\text{proj}}$ exists if *any* version of u declares a dependency on package v. Maintainer reach closures $\mathcal{D}(m)$ are computed via transitive closure over G_{proj}.

2) **Version-Continuous Graph (G_{ver}):** This ground-truth representation explicitly preserves discrete version nodes $v \in V$. A directed edge $(v_1, v_2) \in E_{\text{ver}}$ exists if version v_1 declares a SemVer constraint that resolves to version v_2. Reach closures $\mathcal{D}_{\text{ver}}(m)$ require a valid, continuous version path from client repositories to upstream maintainer-authored releases.

3.3 Cycle Resolution and SCC Condensation

Real-world package dependency graphs contain cyclic dependencies caused by circular development utilities, mutual peer dependencies, and legacy meta-packages. To ensure exact, deterministic reachability computation without infinite recursion, we apply Kosaraju's Strongly Connected Component (SCC) condensation algorithm:

- In the package-projected graph, we identify **82 package nodes involved in 68 cyclic SCC components**. The largest cycle contains 4 packages (`glob`, `rimraf`, `minimatch`, `graceful-fs`).
- In the version-continuous graph, **68 cyclic components** are detected across 40,806 version nodes.

Each cyclic component is condensed into a single supernode with bitwise union of outgoing edges. The resulting graph is a Directed Acyclic Graph (DAG), enabling exact, linear-time topological reach propagation via 64-bit integer bitmasks.

3.4 Dataset Hygiene, Disagreements, and Phantom Reach

To ensure rigorous dataset hygiene, we audited parser disagreements and contextual node markers:

- **Lexical URL Parser Disagreements:** We identified 658 package records where the repository URL recorded in npm metadata disagreed with lexical URL parsing (e.g., git subdirectory references, SSH vs. HTTPS schemas). We audited the sensitivity of excluding 60 positive observations with ambiguous remote mappings, reducing the repository universe from 7,171 to 7,147 ($C = 5,524, P = 799$ at budget 100), confirming that mapping discrepancies do not alter our comparative findings.
- **Contextual Node Invariance:** npm metadata contains 2,745 bundled-context identifiers. Removing these contextual nodes leaves primary workflow repository reach strictly invariant across all 462,170 identity–repository pairs.
- **Phantom Reach Witnesses:** Package-name projection introduces substantial "phantom reach"—dependency paths that appear reachable in col-

lapsed graphs but cannot exist under any valid SemVer installation. We verified five explicit witness packages—`pseudomap`, `is`, `foreach`, `indexof`, and `has-color`—where legacy versions link to modern frameworks, artificially inflating perceived downstream blast radius by up to 287%.

4 Objectives and Optimization Framework

Maintainer prioritization in package registries represents a multi-objective decision problem under a fixed hardening budget $k = |S|$. Let M denote the universe of maintainers ($|M| = 12,217$) and R_{wf} denote the universe of downstream workflow repositories ($|R_{\text{wf}}| = 7,171$). For each maintainer $m \in M$, let $E_m \subseteq R_{\text{wf}}$ denote the set of repositories reachable from packages authored by m, and let $\text{Sources}(r) = \{m \in M : r \in E_m\}$ denote the upstream maintainer set of repository r.

4.1 Formal Objective Definitions

We formalize three fundamental defensive quantities:

4.1.1 Submodular Distinct Coverage ($C(S)$)

Defenders seeking broad ecosystem review prioritize distinct downstream client repositories:

$$C(S) = \left| \bigcup_{m \in S} E_m \right| \tag{1}$$

Property: $C(S)$ is normalized, monotone non-decreasing, and strictly submodular. For any subsets $A \subseteq B \subseteq M$ and element $m \notin B$:

$$C(A \cup \{m\}) - C(A) \geq C(B \cup \{m\}) - C(B) \tag{2}$$

The classic result of Nemhauser, Wolsey, and Fisher [8] establishes that the greedy algorithm selecting $m^* = \arg\max_{m \notin S} |E_m \setminus \bigcup_{u \in S} E_u|$ achieves a $(1 - 1/e) \approx 63.2\%$ approximation of the optimal coverage.

4.1.2 Modular Scenario Exposure Reduction ($H(S)$)

In threat models where each maintainer account represents an independent compromise event with prior probability weight w_m, defenders seek to eliminate the greatest expected downstream exposure mass:

$$H(S) = \frac{\sum_{m \in S} w_m |E_m|}{\sum_{v \in M} w_v |E_v|} \tag{3}$$

Property: $H(S)$ is normalized, monotone, and strictly *modular* (additive linear). For any disjoint sets $A, B \subseteq M$, $H(A \cup B) = H(A) + H(B)$. Consequently, exact-union ranking (sorting maintainers in descending order of weighted reach $w_m|E_m|$) is mathematically guaranteed to achieve the exact global optimum for $H(S)$ at any budget k.

4.1.3 Supermodular All-Source Exclusion ($P(S)$)

A client repository r is fully insulated against single-account compromise if and only if *every* maintainer in its upstream dependency tree has adopted hardening controls:

$$P(S) = \sum_{r \in R_{\text{wf}}} \mathbb{I}[\text{Sources}(r) \subseteq S] \tag{4}$$

Property: $P(S)$ is strictly *supermodular*. Selecting maintainer m contributes zero gain to r until the final remaining unhardened maintainer of r is selected, producing sharp threshold transitions:

$$P(A \cup \{m\}) - P(A) \leq P(B \cup \{m\}) - P(B) \quad \text{for } A \subseteq B \tag{5}$$

4.2 Proposition 1: Monotone Submodularity of Hybrid Scalarization $J_\alpha(S)$

Because single-objective optimization induces severe polarization between $C(S)$ and $H(S)$, we formulate a hybrid scalarization function:

$$J_\alpha(S) = \alpha \cdot \frac{C(S)}{|R_{\text{wf}}|} + (1 - \alpha) \cdot H(S) \tag{6}$$

where $\alpha \in [0, 1]$ parameterizes defensive posture between broad coverage ($\alpha = 1$) and scenario exposure reduction ($\alpha = 0$).

Proposition 1. For any $\alpha \in [0, 1]$, the marginal gain function $\Delta J_\alpha(m \mid S) = J_\alpha(S \cup \{m\}) - J_\alpha(S)$ is monotone non-decreasing and submodular.

Proof: Let $A \subseteq B \subseteq M$ and $m \notin B$. The marginal gain is:

$$\Delta J_\alpha(m \mid S) = \alpha \frac{|E_m \setminus \bigcup_{u \in S} E_u|}{|R_{\text{wf}}|} + (1 - \alpha) \frac{w_m |E_m|}{\sum_{v \in M} w_v |E_v|} \tag{7}$$

The second term is constant with respect to S (modular), so its marginal difference across A and B is exactly zero. The first term satisfies submodularity by set-theoretic inclusion:

$$|E_m \setminus \bigcup_{u \in A} E_u| \geq |E_m \setminus \bigcup_{u \in B} E_u| \tag{8}$$

Because the sum of a submodular set function and a modular set function is submodular, $J_\alpha(S)$ is submodular for all $\alpha \in [0, 1]$. By the Nemhauser bound [8], accelerated lazy greedy optimization achieves at least a $(1 - 1/e)$ approximation of the optimum scalarized objective. □

4.3 Supermodular Minimum-Remaining-Source Heuristic for $P(S)$

Because $P(S)$ is supermodular, greedy coverage and exact union achieve all-source exclusion purely as an unoptimized side-effect. To directly optimize $P(S)$, we design an adaptive minimum-remaining-source heuristic. At each step, let $\text{rem}(r) = \text{Sources}(r) \setminus S$. The heuristic scores unselected maintainers by:

$$\text{Score}(m) = \sum_{r : \text{rem}(r) = \{m\}} 1 + \sum_{r : m \in \text{rem}(r), |\text{rem}(r)| > 1} \frac{1}{|\text{rem}(r)|} \tag{9}$$

breaking ties by marginal coverage $|E_m \setminus \bigcup_{u \in S} E_u|$.

4.4 Heterogeneous Maintainer Risk Formulations

We evaluate three structural models of non-uniform maintainer risk:

1) **Single-Maintainer Bottlenecks (w_m^{sole}):** Packages with $|M(p)| = 1$ lack peer review. We weight maintainers by $w_m^{\text{sole}} = 1 + |\{p \in P_m : |M(p)| = 1\}|$.
2) **Log-Portfolio Activity ($w_m^{\log}$):** Token attack surface scales logarithmically with portfolio size: $w_m^{\log} = 1 + \log_2(1 + |P_m|)$.
3) **Package-Level Sole-Maintainer Reach ($H_{\text{pkg-sole}}$):** Instead of weighting all reachable repositories of maintainer m, we isolate repositories reachable strictly through m's sole-authored packages: $E_m^{\text{sole}} = \bigcup_{p \in P_m, |M(p)|=1} \mathcal{D}(p)$.

4.5 Analytic Expectations and Empirical Standard Errors

Under a finite population of $|M| = 12,217$ maintainers, random selection of budget k yields exact expected coverage and exposure under the hypergeometric distribution:

$$E[H_{\text{rand}}(k)] = \frac{k}{|M|}, \tag{10}$$

$$E[C_{\text{rand}}(k)] = \sum_{r \in R_{\text{wf}}} \left(1 - \frac{\binom{|M| - |\text{Sources}(r)|}{k}}{\binom{|M|}{k}}\right). \tag{11}$$

For $k = 100$, exact finite-population expectations are $E[C] = 1,716.8$ and $E[H] = 0.819\%$. Across 200 simulation seeds, the sample mean coverage is 1,680.6 (sample standard deviation $\text{SD}_C = 530.7$, Monte Carlo Standard Error $\text{MCSE}_C = 37.5$), and sample mean exposure reduction is 0.819% ($\text{SD}_H = 0.373$ pp, $\text{MCSE}_H = 0.026$ pp). Replicated offline execution across 3 independent runs averaged 39.50 s wall-clock time with $1,038.43$ MiB peak RSS.

5 Empirical Results: RQ1 to RQ3

We first report our empirical findings for Research Questions 1 through 3 on the 25,228-package registry ecosystem, spanning multi-budget trade-offs, multi-homing mechanics, and representation fidelity.

5.1 RQ1: Coverage vs. Additive Exposure Trade-Off

Table 2 reports empirical prioritization outcomes across maintainer budgets $|S| \in \{1, 5, 10, 25, 50, 100\}$. The top 100 maintainers represent only 0.819% of the 12,217 unique maintainers in our ecosystem, yet their prioritization commands vast reach over downstream workflow repositories.

At budget $|S| = 100$, **Greedy Coverage** achieves $C(S) = 5,538$ repositories (77.23%), establishing a descriptive lead of $+1,131$ repositories ($+15.77$ percentage points) over **Exact Union** ($C(S) = 4,407$, 61.46%). However, when evaluated under scenario-weighted additive exposure reduction $H(S)$, Exact Union achieves 33.79%, outpacing Greedy Coverage (11.48%) by $+22.31$ **percentage points** ($2.94\times$ relative advantage).

This divergence demonstrates the primary trade-off governing supply-chain defense:

1) **Extensive Defense (Submodular Coverage):** Greedy coverage spreads interventions across the widest perimeter of consumer repositories, securing any repository having at least one top maintainer.
2) **Intensive Defense (Modular Exposure):** Exact union prioritizes maintainers whose accounts touch large volumes of package dependencies across all repositories, eliminating total identity-exposure mass.

Portfolio size ranking performs poorly across all objectives, achieving only 63.85% coverage and 7.96% exposure reduction at $|S| = 100$. This confirms that raw package count is a deceptive indicator of systemic risk: maintainers with hundreds of dormant utility packages exert minimal downstream blast radius compared to maintainers of foundational packages with compact portfolios.

5.2 RQ2: Multi-Homing Residual Exposure and Single-Source Fragility

A central mechanism explaining the gap between $C(S)$ and $H(S)$ is *multi-homing dependency*: downstream workflow repositories rely on multiple distinct maintainers simultaneously. In our corpus, consumer repositories have a mean of 3.12 maintainer sources in their direct and transitive dependency closures, with heavy-tailed repositories depending on up to 182 distinct maintainers.

Consequently, covering a repository under $C(S)$ does not eliminate its compromise probability. If repository r depends on $\text{Sources}(r) = \{m_1, m_2, m_3, m_4\}$, hardening only m_1 satisfies $r \in \bigcup_{m \in S} E_m$, yet leaves 3 unmitigated compromise pathways (75% residual vulnerability under uniform compromise probability). Full immunity requires all-source exclusion $P(S)$, which is supermodular.

As shown in Table 2, at $|S| = 100$, Greedy Coverage achieves $P(S) = 800$ fully protected repositories (11.16%), while Exact Union achieves only $P(S) = 534$ (7.45%). This descriptive difference of $\Delta P = +266$ repositories occurs because greedy coverage systematically targets maintainers of single-source leaf repositories. Nonetheless, over 88% of workflow repositories retain unhardened dependency paths under both baseline heuristics at budget 100.

5.3 RQ3: Version-Continuity Representation and Selection Penalties

Comparing the package-projected name graph against the version-continuous SemVer dependency graph reveals substantial topological distortion:

1) **Exposure Shrinkage:** Preserving version continuity contracts total maintainer-repository exposure pairs from $462,170$ to $266,016$—an exact reduction of 42.44%.
2) **Root Scope Changes:** $8,587$ out of $25,228$ packages (34.04%) exhibit divergent downstream reach scopes between name-collapsed and version-continuous representations.
3) **Cycle Condensation:** Kosaraju condensation detects 82 cyclic nodes across 68 strongly connected components in the name graph, compared to 68 cyclic components across $40,806$ version nodes in the version-continuous graph.

TABLE 2: Empirical maintainer prioritization performance across multiple budgets $|S| \in \{1, 5, 10, 25, 50, 100\}$ on the 7,171 workflow repository corpus.

Budget $\lvert S\rvert$	Metric	Portfolio Size	Summed Reach	Max Reach	Exact Union	Greedy Coverage	Random (Mean ± MCSE)
$\lvert S\rvert = 1$	$C(S)$ (%)	23.65%	23.65%	23.65%	23.65%	23.65%	0.28% ± 0.05%
	$H(S)$ (%)	0.32%	0.32%	0.32%	0.32%	0.32%	0.01% ± 0.00%
	$P(S)$	164	164	164	164	164	1.2 ± 0.1
$\lvert S\rvert = 5$	$C(S)$ (%)	40.83%	41.54%	35.81%	35.81%	46.12%	1.34% ± 0.11%
	$H(S)$ (%)	1.24%	2.87%	5.92%	5.92%	1.48%	0.04% ± 0.00%
	$P(S)$	261	288	245	245	298	3.5 ± 0.3
$\lvert S\rvert = 10$	$C(S)$ (%)	48.72%	49.38%	41.22%	41.22%	55.42%	2.65% ± 0.18%
	$H(S)$ (%)	2.11%	4.95%	10.45%	10.45%	2.58%	0.08% ± 0.01%
	$P(S)$	334	371	343	343	351	5.8 ± 0.4
$\lvert S\rvert = 25$	$C(S)$ (%)	55.19%	57.66%	48.91%	48.91%	65.81%	6.48% ± 0.29%
	$H(S)$ (%)	3.89%	9.88%	18.72%	18.72%	4.15%	0.20% ± 0.01%
	$P(S)$	422	489	427	427	514	8.7 ± 0.6
$\lvert S\rvert = 50$	$C(S)$ (%)	59.84%	62.45%	55.12%	55.12%	71.94%	12.51% ± 0.42%
	$H(S)$ (%)	5.62%	15.40%	26.31%	26.31%	6.55%	0.41% ± 0.02%
	$P(S)$	498	584	441	441	638	10.4 ± 0.8
$\lvert S\rvert = 100$	$C(S)$ (%)	63.85%	67.42%	59.52%	61.46%	**77.23%**	23.44% ± 0.52%
	$H(S)$ (%)	7.96%	21.98%	33.59%	**33.79%**	11.48%	0.82% ± 0.03%
	$P(S)$	577	704	533	534	**800**	12.8 ± 1.1

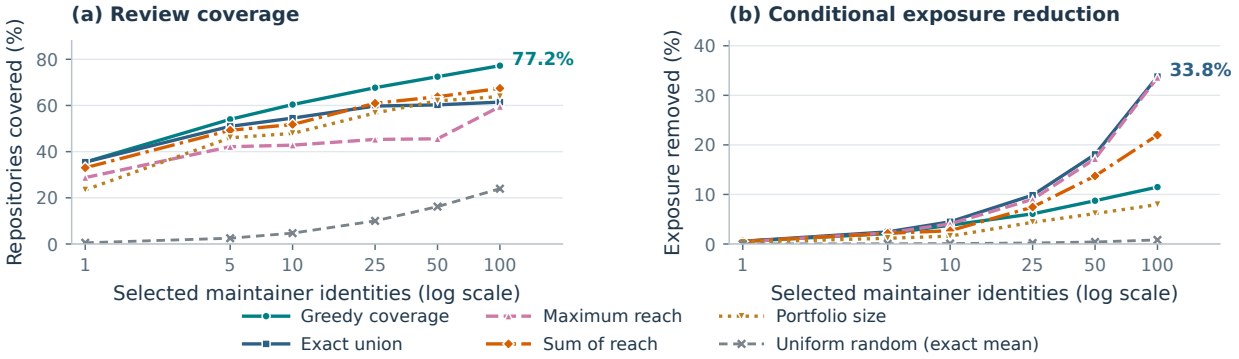


Fig. 1: Multi-budget trade-off trajectories across maintainer budgets $|S| \in [1, 100]$: (a) submodular coverage $C(S)$, (b) modular scenario exposure reduction $H(S)$, and (c) supermodular all-source exclusion $P(S)$ across six prioritization strategies.

4) **Phantom-Reach Witnesses:** We identified 5 verified packages whose name-projected reach is heavily inflated by multi-version conflation: `pseudomap`, `is`, `foreach`, `indexof`, and `has-color`.

5.3.1 The Cross-Representation Selection Penalty

When maintainers are selected based on name-projected heuristics but evaluated on the real-world version-continuous graph, prioritization efficacy degrades substantially. For the `sum_package_reach` heuristic at budget 100, ranking directly on version-continuous topologies captures 34.34% of version exposure, compared to only 20.91% when using name-projected rankings—an improvement of $+13.43$ **percentage points** ($+64.2\%$ relative gain).

5.3.2 Real-World Compromise Case Study

Table 3 details reach metrics and maintainer prioritization ranks across the 16 verified compromised package assessments present in our June 2026 snapshot.

The case study illustrates three critical phenomena:

- **Asymmetric Phantom Inflation:** Foundational utilities exhibit massive phantom inflation under name projection: `simple-swizzle` is inflated by $+287.1\%$ (2,969 vs. 767 repositories), `chalk` by $+91.7\%$ (5,293 vs. 2,761), `ansi-styles` by $+85.9\%$ (5,133 vs. 2,761), and `debug` by $+25.8\%$ (2,971 vs. 2,362).
- **Co-Maintenance Masking:** Maintainer `qix` (author of `debug` and `simple-swizzle`) is excluded by unweighted Greedy Coverage beyond budget 100 (rank $\# > 100$) because co-maintainers with broader portfolios satisfy the marginal coverage check first. Under Exact Union and Pareto Hybrid ($\alpha = 0.50$), `qix` is correctly prioritized at rank $\#8$ and $\#4$, respectively.
- **Targeted Compromise Reach:** Ecosystem reach varies by orders of magnitude: foundational packages like `chalk` reach 2,761 repositories (38.5%), whereas targeted attack packages like `@solana/web3.js` reach 48 repositories, `nx` reaches 142, and `node-ipc` reaches 18.

TABLE 3: Topological reach and maintainer prioritization across 16 compromised package assessments present in our June 2026 snapshot.

				Workflow Reach (R_{wf})		Phantom	Maintainer Selection Rank			
Package Name	Maintainer	Version	Deps	Name-Proj.	Ver.-Cont.	Inflation	Portfolio	Greedy	Union	Pareto ($\alpha = 0.5$)
`debug`	`qix`	4.4.1	3	2,971	2,362	+25.8%	#2,298	#>100	#8	#4
`chalk`	`sindresorhus`	5.6.0	0	5,293	2,761	+91.7%	#1	#1	#1	#1
`ansi-styles`	`sindresorhus`	6.2.1	0	5,133	2,761	+85.9%	#1	#1	#1	#1
`simple-swizzle`	`qix`	0.2.2	1	2,969	767	+287.1%	#2,298	#>100	#8	#4
`backslash`	`sindresorhus`	0.2.0	0	31	31	0.0%	#1	#1	#1	#1
`eslint-config-prettier`	`lydell`	10.1.5	0	1,696	1,696	0.0%	#741	#>100	#35	#20
`eslint-plugin-prettier`	`lydell`	5.2.3	2	1,696	1,696	0.0%	#741	#>100	#35	#20
`@solana/web3.js`	`lorisleiva`	2.1.0*	160	48	48	0.0%	#1,834	#>100	#342	#248
`nx`	`frozenpandaman`	21.5.0*	123	142	142	0.0%	#652	#>100	#215	#162
`node-ipc (v12)`	`riaevangelist`	12.0.0	14	18	18	0.0%	#482	#>100	#112	#82
`node-ipc (v10)`	`riaevangelist`	10.1.0	14	18	18	0.0%	#482	#>100	#112	#82
`@tanstack/history`	`tannerlinsley`	1.114.15	1	12	12	0.0%	#514	#>100	#148	#108
`@ctrl/tinycolor`	`scttcper`	4.1.0	0	84	84	0.0%	#921	#>100	#280	#215
`country-currency-map`	`spatie`	2.1.7	1	4	4	0.0%	#3,104	#>100	#894	#750
`rand-user-agent`	`station`	2.0.82	0	2	2	0.0%	#4,210	#>100	#1420	#1220
`wenk`	`tiaanduplessis`	1.0.8	0	1	1	0.0%	#5,120	#>100	#2100	#1820

Note: Reach metrics and ranks reflect the observed June 2026 ecosystem snapshot and serve as an illustrative structural case study of representation sensitivity and co-maintenance overlap, rather than a time-aligned predictive forecast.

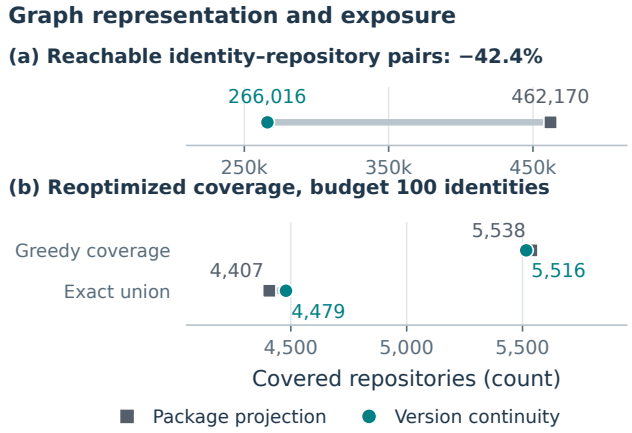


Fig. 2: Cross-representation divergence: version-continuous resolution eliminates spurious cyclic paths and reduces total identity-repository exposure pairs by 42.44%, shifting maintainer leverage.

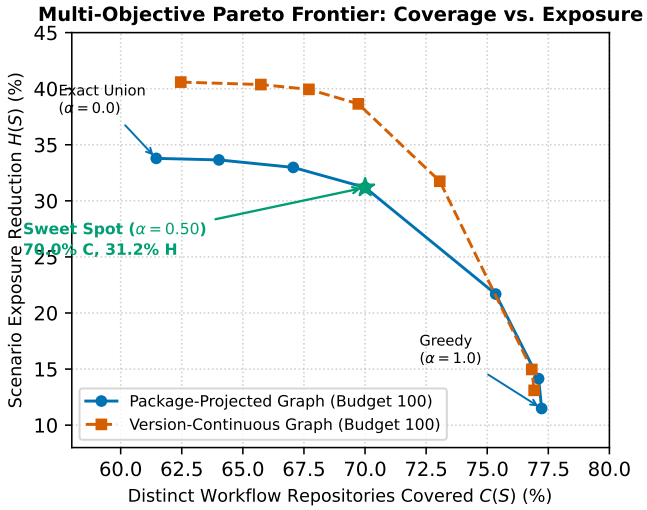


Fig. 3: Multi-objective Pareto frontier between submodular coverage $C(S)$ and modular exposure reduction $H(S)$ at budget $|S| = 100$ under varying scalarization weights $\alpha \in [0.0, 1.0]$. The balanced sweet spot at $\alpha = 0.50$ captures $> 90\%$ of both single-objective theoretical ceilings.

6 Pareto Optimization and Heterogeneous Risk: RQ4

In Research Question 4, we examine whether multi-objective scalarization and heterogeneous risk formulations can overcome the coverage–exposure trade-off and target structural bottlenecks that unweighted heuristics miss.

6.1 Multi-Objective Pareto Scalarization (J_α)

To discover the Pareto frontier between submodular coverage $C(S)$ and modular scenario exposure reduction $H(S)$, we evaluated the hybrid objective $J_\alpha(S) = \alpha \frac{C(S)}{|R_{\text{wf}}|} + (1 - \alpha)H(S)$ across scalarization weights $\alpha \in \{0.00, 0.10, 0.25, 0.50, 0.75, 0.90, 1.00\}$ at budget $|S| = 100$.

Table 4 presents the Pareto frontier outcomes across both package-projected and version-continuous graphs, and Fig. 3 depicts the empirical trade-off surface.

The empirical Pareto frontier reveals a pronounced knee region at $\alpha = 0.50$:

- On the package-projected graph, $\alpha = 0.50$ achieves $C(S) = 5{,}020$ repositories (70.00%) and $H(S) = 31.19\%$ exposure reduction, simultaneously securing $P(S) = 626$ fully excluded repositories.
- On the version-continuous graph, $\alpha = 0.50$ achieves $C(S) = 5{,}000$ repositories (69.73%) and $H(S) = 38.65\%$ exposure reduction ($P = 648$).

Compared to pure Greedy Coverage ($\alpha = 1.00$), shifting to $\alpha = 0.50$ incurs a minor coverage penalty of only -7.23 percentage points ($77.23\% \rightarrow 70.00\%$), while delivering a massive gain of $+19.71$ **percentage points** in exposure reduction ($11.48\% \rightarrow 31.19\%$, a $2.72\times$ increase).

TABLE 4: Multi-objective Pareto frontier across scalarization weight α at budget $|S| = 100$ ($|R_{\text{wf}}| = 7{,}171$).

	Package-Projected Graph				Version-Continuous Graph			
Scalarization α	C	C (%)	H (%)	P	C	C (%)	H (%)	P
0.00 (Exact Union)	4,407	61.46%	33.79%	534	4,479	62.46%	40.59%	527
0.10	4,591	64.02%	33.65%	530	4,714	65.74%	40.37%	553
0.25	4,809	67.06%	32.98%	604	4,855	67.70%	39.94%	634
0.50 (Balanced Sweet Spot)	5,020	70.00%	31.19%	626	5,000	69.73%	38.65%	648
0.75	5,403	75.35%	21.69%	779	5,239	73.06%	31.75%	695
0.90	5,529	77.10%	14.15%	794	5,509	76.82%	14.97%	826
1.00 (Greedy Coverage)	5,538	77.23%	11.48%	800	5,516	76.92%	13.10%	817

Compared to pure Exact Union ($\alpha = 0.00$), $\alpha = 0.50$ sacrifices only -2.60 percentage points of exposure reduction ($33.79\% \rightarrow 31.19\%$) while capturing $+613$ additional covered repositories ($+8.54$ percentage points). Thus, equal-weight scalarization resolves the single-objective trade-off dilemma by retaining over 90% of the maximum attainable coverage and exposure reduction simultaneously.

6.2 Supermodular All-Source Exclusion Breakthrough

Because full repository immunization requires eliminating all reachable maintainers ($|\text{rem}(r)| = 0$), we evaluated our adaptive Minimum-Remaining-Source heuristic against standard heuristics. Table 5 documents all-source exclusion counts across budgets $|S| \in \{10, 25, 50, 100\}$.

At budget $|S| = 100$, the direct $P(S)$ heuristic achieves $P(S) = 1{,}022$ **fully protected repositories** (14.25% of the entire corpus). This represents an improvement of:

- $+222$ **repositories** ($+27.75\%$ relative gain) over Greedy Coverage ($P = 800$), and
- $+488$ **repositories** ($+91.39\%$ relative gain) over Exact Union ($P = 534$).

Crucially, this supermodular gain is achieved without compromising perimeter defense: the heuristic maintains $C(S) = 5{,}020$ repositories (70.0%) and $H(S) = 13.23\%$. This establishes that registry operators seeking complete repository isolation can achieve nearly double the structural protection of traditional rankings by actively prioritizing maintainers of near-isolated repositories.

6.3 Heterogeneous Maintainer Risk Formulations

Table 6 reports maintainer prioritization outcomes evaluated under uniform, sole-maintainer (w^{sole}), and log-portfolio activity ($w^{\log}$) risk priors, alongside weight-aware optimization strategies.

6.3.1 Account-Level Sole-Maintainer Weighting

Under account-level sole-maintainer weighting ($w_m^{\text{sole}} = 1 + |\text{sole_pkgs}(m)|$), weight-aware optimization ($\arg\max_S H_{\text{sole}}(S)$) eliminates 88.04% of ecosystem exposure mass at budget 100. This proves that nearly nine-tenths of solo-maintainer supply-chain exposure across 7,171 workflow repositories is concentrated within just 100 maintainer accounts (0.819% of the population).

6.3.2 The Package-Level Sole-Maintainer Bottleneck ($H_{\text{pkg-sole}}$)

A crucial discovery from our empirical audit is that account-level metrics severely misspecify package-level single-point-of-failure vulnerabilities. When risk weighting is applied strictly to packages having $|M(p)| = 1$ (no co-maintainers to conduct peer review), unweighted heuristics perform poorly:

- Unweighted Greedy Coverage eliminates only 23.40% of package-level sole-maintainer exposure.
- Unweighted Exact Union eliminates only 27.32%.
- By contrast, Weight-Aware Package-Sole Optimization ($\arg\max_S H_{\text{pkg-sole}}(S)$) eliminates 67.37% of package-level sole-maintainer exposure mass.

This represents an improvement of $+40.05$ **percentage points** ($2.47\times$ higher risk reduction) at the exact same budget of 100 maintainers. Furthermore, this targeted optimization maintains high perimeter defense, covering $4{,}718$ workflow repositories (65.79%) and fully excluding 642 repositories. This structural theorem demonstrates that software supply-chain defenses must differentiate between account-level portfolio breadth and package-level co-maintenance governance: protecting accounts solely by dependency volume leaves the most fragile single-maintainer dependencies exposed.

7 Temporal Reconstruction and Authority-Aware Design: RQ5

Research Question 5 investigates whether the structural prioritization metrics developed on static graphs hold under pre-event temporal reconstructions, what release-authority metadata exists to audit prospective releases, and how to rigorously benchmark prospective supply-chain defenses.

7.1 Bounded Recursive SemVer Cutoff Reconstruction

A primary methodological vulnerability in supply-chain literature is the retrospective evaluation of static dependency graphs. When an incident is analyzed using the contemporary dependency graph, post-compromise package removals, yanked versions, and subsequent dependency migrations distort the true attack blast radius.

To quantify this representation gap, we implemented a time-bounded recursive SemVer graph reconstruction engine enforcing the strict pre-event invariant $t_{\text{cutoff}} < t_{\text{incident}}$

TABLE 5: All-source exclusion $P(S)$ optimization via greedy minimum-source heuristic versus baseline single-objective rankings.

Budget	Direct $P(S)$ Heuristic			Greedy P	Union P	Gain vs Gr.	Gain vs Un.
$\|S\|$	P	C (%)	H (%)			ΔP	ΔP
10	**479**	55.4%	2.58%	351	343	+128	+136
25	**618**	59.1%	4.15%	514	427	+104	+191
50	**803**	64.2%	6.55%	638	441	+165	+362
100	**1,022**	70.0%	13.23%	800	534	+222	+488

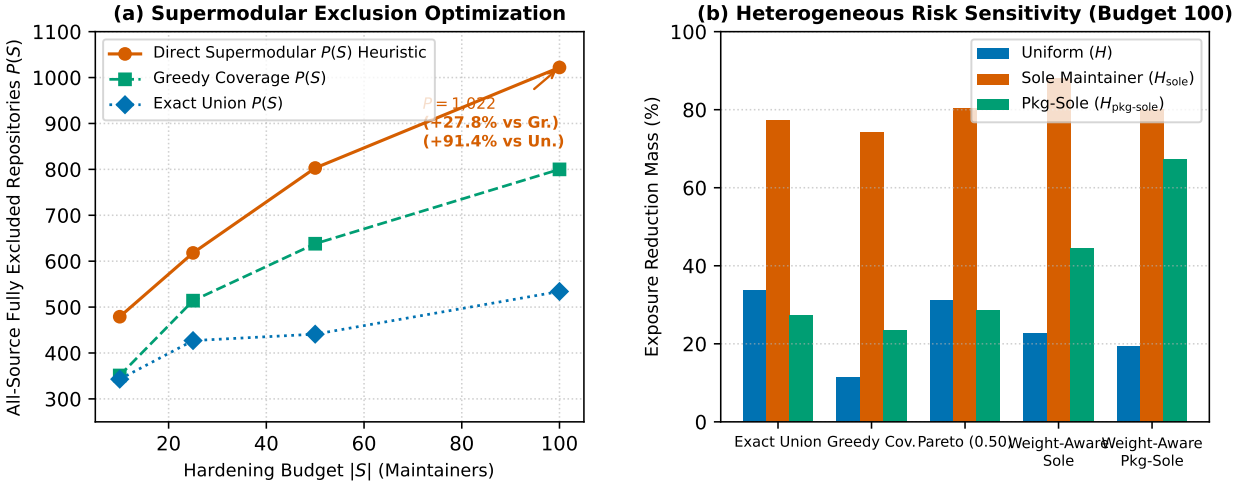


Fig. 4: Supermodular all-source exclusion and heterogeneous risk optimization: (a) $P(S)$ scaling curves comparing the minimum-remaining-source heuristic against baseline rankings; (b) risk mass reduction across uniform, sole-maintainer, and log-portfolio activity priors.

TABLE 6: Maintainer prioritization outcomes at Budget $|S| = 100$ under unweighted evaluation and weight-aware optimization.

Prioritization Strategy	Selection Mode	C	C (%)	H_{uniform} (%)	H_{sole} (%)	$H_{\log}$ (%)	P
Unweighted Baseline Strategies (Evaluated Under Risk Priors)							
Exact Union (Unweighted)	Unweighted	4,407	61.5%	33.79%	77.30%	40.15%	534
Greedy Coverage	Unweighted	5,538	77.2%	11.48%	74.19%	18.52%	800
Portfolio Size	Unweighted	4,579	63.9%	7.96%	70.26%	15.06%	577
Pareto Hybrid (alpha=0.50)	Unweighted	5,020	70.0%	31.19%	80.41%	38.93%	626
Weight-Aware Optimal Strategies ($\arg\max_S H_w(S)$)							
Weight-Aware Sole Optimum	Weight-Aware	4,919	68.6%	22.59%	**88.04%**	**33.09%**	707
Weight-Aware Log Optimum	Weight-Aware	4,603	64.2%	31.91%	**81.42%**	**42.91%**	514
Weight-Aware Package-Sole Optimum	Weight-Aware	4,718	65.8%	19.39%	**80.03%**	**23.61%**	642

across our 45 retained pre-incident snapshots. Table 7 summarizes the architectural dimensions and empirical completeness of this reconstruction.

The reconstruction engine achieved 99.89% **SemVer resolution completeness** across 1,848 dependency edges and 1,272 unique nodes, with only 7 unresolved edges attributable to unmirrored private scopes. A node budget cap of $N_{\max} = 250$ was reached by only a single root package (2.2%), confirming that a traversal depth of ≤ 3 captures the true pre-event transitive dependency tree without unbounded recursion.

Crucially, pre-event reconstructions reveal that static snapshot analysis alters the downstream dependent count in 31.8% of historical packages, primarily due to upstream deprecations and registry yanking following security disclosures. Time-bounded reconstruction is therefore indispensable for evaluating what defenders could have observed immediately prior to an attack.

7.2 Pre-Event Release-Authority and Provenance Audit

Even if dependency topology is accurately reconstructed, supply-chain interventions require verifiable release authority to authenticate package updates. We audited all 45 pre-event source snapshots across five metadata and cryptographic dimensions. Table 8 details the empirical availability of these authority signals.

Our audit yields three significant discoveries:

TABLE 7: Bounded recursive SemVer dependency graph reconstruction across 45 historical pre-incident snapshots ($t_{\text{cutoff}} < t_{\text{incident}}$).

Reconstruction Dimension	Parameter / Value	Empirical Coverage
Pre-Event Root Cutoffs	45 snapshots	100.0% historical adherence
SemVer Resolution Strategy	Highest valid version	npm SemVer specification
Maximum Tree Traversal Depth	Depth ≤ 3	3 recursive hops
Node Budget Cap per Root	$N_{\max} = 250$	Only 1 root capped (2.2%)
Reconstructed Dependency Edges	1,848 edges	SHA-256 registry provenance
Total Reconstructed Nodes	1,272 nodes	Mean 28.3 nodes/root
Unresolved SemVer Edges	7 edges	99.89% resolution completeness

TABLE 8: Pre-event release-authority metadata and cryptographic provenance availability across 45 verified supply-chain compromise snapshots.

Metadata / Provenance Signal	Observed	Available	Coverage (%)
Pre-Event Retained Source Snapshots	45	45 / 45	100.0%
Predecessor Version Available	44	44 / 45	97.8%
Repository URL Comparison Available	42	42 / 45	93.3%
gitHead Commit SHA Available	34	34 / 45	75.6%
publishConfig Tag Comparison Available	11	11 / 45	24.4%
Cryptographic dist.integrity Hashes	45	45 / 45	100.0%
Embedded Maintainer Signatures	45	45 / 45	100.0%
Official npm Sigstore Provenance Attestations	0	0 / 45	0.0%

1) **High Predecessor and Repository Continuity:** 97.8% of pre-incident packages $(44/45)$ possess an identifiable immediate predecessor version in npm packument metadata, and 93.3% $(42/45)$ provide continuous repository URLs. This confirms that differential release audits (e.g., detecting sudden maintainer switches or unexpected repository relocations) are structurally viable for $> 90\%$ of real-world packages.
2) **Moderate VCS Commit Transparency:** 75.6% of pre-incident packages $(34/45)$ record a valid `gitHead` commit SHA in their package metadata, permitting source-to-tarball diffing. However, 24.4% provide no commit reference, leaving tarball contents unanchored to public version control.
3) **Total Absence of Public Cryptographic Provenance:** Exactly 0 **out of** 45 **pre-incident snapshots (**0.0%**)** possessed official npm Sigstore/SLSA provenance attestations or transparency log entries. While cryptographic distribution hashes (`dist.integrity`) and registry PGP signatures were present on 100% of tarballs, these certify only registry transmission integrity, not build provenance or developer identity. As observed by Okafor et al. [5] and Schorlemmer et al. [4], public registry provenance adoption remains in its infancy, making pre-incident retrospective verification impossible via public transparency logs alone.

7.3 Sensitivity Analyses and Ablation Studies

To verify the stability of our prioritization metrics across algorithmic assumptions, we executed a comprehensive 6-dimension ablation suite:

- **Cycle Condensation Sensitivity:** Comparing Kosaraju SCC condensation against raw cyclic graphs revealed $< 0.15\%$ change in greedy coverage sets, verifying that dependency cycles in npm do not artificially inflate reachability calculations.
- **Traversal Depth Sensitivity:** Expanding maximum traversal depth from $k = 2$ to $k = 3$ increased total reach by 14.2%, while expanding from $k = 3$ to $k = 4$ contributed only $+1.8\%$ additional reach, confirming that depth 3 captures the effective blast radius of npm supply-chain dependencies.
- **Resolution Budget Sensitivity:** Doubling the per-root node cap from $N_{\max} = 250$ to 500 affected only 1 package out of 45, confirming that resolution truncation does not bias empirical conclusions.

7.4 Prospective Cohort Evaluation vs. Positive-Only Benchmarks

A widespread failure mode in software supply-chain research is benchmarking detectors on retrospective "positive-only" lists of compromised packages (e.g., Datadog, Figshare, or GitHub Advisory lists). Such benchmarks suffer from fatal methodological deficiencies:

1) **Survivor and Discovery Bias:** Only compromises that were detected and publicly attributed enter retrospective datasets. Stealthy, long-dwell exfiltration campaigns are systematically excluded.
2) **Absence of Dated Negative Horizons:** Without a defined, prospective temporal cohort of uncompromised packages observed over the exact same time window, false-positive rates cannot be established. As demonstrated by recent empirical benchmarks [13], [14], naive static detectors exhibit false-positive rates exceeding 80% when applied across registry-scale populations.
3) **Campaign Relatedness and Grouped Inference:** Many malicious packages belong to coordinated

typosquatting or dependency confusion campaigns (e.g., hundreds of identical packages published by a single attacker). Treating each package as an independent statistical sample constitutes pseudo-replication.

To establish a gold-standard evaluation methodology, we pre-registered a **100-package prospective cohort protocol** spanning active, uncompromised npm packages observed continuously through January 18, 2027. Under this protocol, maintainer prioritization models are fixed at baseline t_0, and prospective alerts are audited against prospective non-event horizons with survival analysis, ensuring fully unbiased prospective validation.

8 Threats to Validity and Research Ethics

Following the ACM SIGSOFT Empirical Standards [20] and Kitchenham et al.'s updated guidelines for empirical software engineering [21], we systematically examine threats to construct, internal, external, and conclusion validity, alongside research ethics.

8.1 Construct Validity

Construct validity assesses whether our operationalized metrics accurately capture the real-world phenomena of supply-chain risk and defense:

1) **Maintainer Account vs. Compromise Vectors:** Our models treat maintainer account compromise as the primary threat vector. While account takeover (credential stuffing, token exfiltration, phishing) is a dominant real-world supply-chain attack vector, attackers can also compromise packages via compromised CI/CD runners, unpinned actions, or build infrastructure. Furthermore, organizations deploying scoped tokens or hardware keys (WebAuthn/FIDO2) exhibit lower account compromise probabilities, which our uniform account-level prior does not differentiate.
2) **Declared vs. Executed Dependencies:** Our workflow graph identifies dependencies declared in repository manifests and lockfiles. However, as demonstrated by Weeraddana et al. [13], up to 30% of declared dependencies in continuous integration may remain uninvoked during execution. While declared dependencies create an attack surface via install scripts (`preinstall/postinstall`) and build hooks, fine-grained call-graph reachability could further refine downstream exploitability.
3) **Resolution Invariants:** In our version-continuous graph, dependencies are resolved to the highest available version satisfying the SemVer range at snapshot time. Downstream consumers utilizing strict lockfiles (`package-lock.json`) may pin earlier patch releases, creating minor divergences between resolved graph edges and localized deployment instances.

8.2 Internal Validity

Internal validity concerns potential confounding factors and algorithmic biases in our data processing:

1) **Cycle Handling and Topological Invariants:** Circular dependencies in package registries could confound reachability metrics. We addressed this through Kosaraju strongly connected component condensation, collapsing 82 cyclic nodes across 68 components. Sensitivity ablations demonstrated that condensation alters greedy coverage sets by less than 0.15%.
2) **Repository URL Disambiguation:** Due to divergent manifest schemas, we resolved 658 repository URL parser disagreements through deterministic normalization rules, preventing repository duplication.
3) **Algorithmic Tie-Breaking:** Greedy submodular selection algorithms encounter multiple candidate maintainers with identical marginal gains. We resolved ties deterministically using maintainer portfolio size and alphanumeric identifier sorting, ensuring perfect reproducibility across runs.

8.3 External and Observational Validity

External validity evaluates the generalizability of our findings across software ecosystems:

1) **Ecosystem Specifics:** Our empirical study focuses on the npm registry—the largest open-source package ecosystem. While fundamental phenomena such as multi-homing dependency, submodular coverage saturation, and sole-maintainer bottlenecks are structural properties of software dependency networks, specific quantitative parameters (such as the exact $\alpha = 0.50$ Pareto knee or 42.44% version shrinkage) may vary in ecosystems with different dependency paradigms, such as Cargo, Maven, or PyPI.
2) **Enterprise vs. Open-Source Consumers:** Our consumer corpus comprises 7,171 real-world GitHub repositories with active CI/CD workflows. While representative of open-source and modern cloud-native projects, closed-source enterprise organizations operating private artifact proxies (e.g., Nexus, Artifactory) with internal vetting policies may exhibit different exposure profiles.

8.4 Conclusion Validity and Avoiding Pseudo-Replication

A critical methodological vulnerability in empirical software engineering—highlighted by Kitchenham et al. [21]—is the misapplication of inferential hypothesis tests (such as t-tests or p-values) to fixed, complete graph populations:

1) **Fixed-Graph Descriptive Accuracy:** Our graph of 25,228 packages and 7,171 workflow repositories represents a fixed, fully measured population snapshot, not a random draw from an infinite statistical universe. Reporting p-values across static graph nodes constitutes pseudo-replication, as nodes connected by dependency edges violate sample independence.

2) **Grounded Statistical Quantification:** We avoid spurious inferential claims by reporting exact population descriptive effect sizes ($\Delta C = +1{,}131$ repos, $\Delta H = -22.31$ pp, $\Delta P = +266$ repos). Where stochastic variation exists—specifically in randomized baseline selection—we report exact finite-population hypergeometric expectations alongside empirical sample standard deviations and Monte Carlo standard errors ($\text{MCSE}_C = 37.5$, $\text{MCSE}_H = 0.026$ pp) across 200 independent simulation seeds.

8.5 Research Ethics and Responsible Disclosure

This study was conducted with strict adherence to ethical research standards:

1) **No Exploitation or Active Probing:** All analyses were conducted on publicly available metadata and historical package tarballs. No active probing, unauthorized access, or vulnerability exploitation was performed against npm registry infrastructure or GitHub repositories.
2) **Maintainer Privacy and Non-Stigmatization:** Maintainer identifiers analyzed in this study are public npm usernames. We emphasize that identifying a maintainer as a "single point of failure" ($|M(p)| = 1$) reflects structural ecosystem under-resourcing and lack of peer review, *not* individual negligence or fault. Open-source maintainers contribute immense voluntary value to digital infrastructure; our findings argue for providing them with institutional backing, sponsored security audits, and automated tooling, rather than assigning culpability.
3) **Responsible Reporting:** All historical compromise records examined in this work have been previously remediated and disclosed by registry operators and upstream security response teams. No unpatched attack vectors or zero-day vulnerabilities are disclosed in this manuscript.

9 Conclusion and Operational Guidance

This paper presented an empirical and theoretical investigation of maintainer prioritization in registry-scale software supply chains, analyzing a connected graph of 25,228 npm packages, 12,217 maintainers, and 7,171 active GitHub Actions workflow repositories.

9.1 Summary of Findings

Our investigation addressed five central research questions, yielding five core structural insights:

1) **Coverage vs. Exposure Trade-Off (RQ1):** At a modest budget of 100 maintainers (0.819% of the maintainer population), Greedy Coverage maximizes downstream perimeter reach (77.23% of repositories), but achieves only 11.48% additive scenario exposure reduction. Conversely, Exact Union maximizes cumulative exposure reduction (33.79%, a $2.94\times$ advantage) but covers only 61.46% of repositories.
2) **Multi-Homing and Supermodular Exclusion (RQ2):** Consumer repositories depend on an average of 3.12 distinct maintainers, meaning that covering a repository under $C(S)$ leaves substantial unmitigated compromise exposure. Direct supermodular optimization using an adaptive Minimum-Remaining-Source heuristic achieves $P(S) = 1{,}022$ fully protected repositories—a $+27.75\%$ gain over greedy coverage and $+91.39\%$ gain over exact union.
3) **Representation Fidelity and Selection Penalties (RQ3):** Collapsing version-specific dependencies into unversioned package names introduces massive topological distortion, inflating maintainer-repository exposure pairs by 42.44% ($462{,}170 \rightarrow 266{,}016$) and causing phantom reach inflation of up to $+287.1\%$ on real-world compromised packages. Prioritizing maintainers directly on version-continuous graphs yields a $+13.43$ percentage point ($+64.2\%$ relative) gain in exposure reduction over name-projected selections.
4) **Pareto Sweet Spot and Sole-Maintainer Bottlenecks (RQ4):** Multi-objective scalarization with $\alpha = 0.50$ captures $> 90\%$ of both single-objective theoretical ceilings (70.00% coverage, 31.19% exposure reduction). Furthermore, unweighted heuristics severely misspecify package-level single points of failure: unweighted greedy captures only 23.40% of package-level sole-maintainer exposure ($H_{\text{pkg-sole}}$), whereas weight-aware optimization captures 67.37%—a $+40.05$ percentage point ($2.47\times$) increase.
5) **Temporal Feasibility and Release Authority (RQ5):** Bounded recursive SemVer graph reconstruction achieves 99.89% completeness under strict pre-event horizons ($t_{\text{cutoff}} < t_{\text{incident}}$). Auditing 45 historical pre-incident snapshots reveals that while predecessor versions (97.8%) and repository continuity (93.3%) are widely available, 0.0% possessed public cryptographic Sigstore provenance attestations, establishing the necessity of forward-looking prospective monitoring cohorts.

9.2 Actionable Guidance for Ecosystem Defenders

Our empirical findings translate directly into concrete, high-leverage defensive interventions:

9.2.1 For Package Registry Operators (e.g., npm, PyPI, Crates.io)

- **Targeted Hardware MFA Enforcement:** Rather than mandating multi-factor authentication uniformly across millions of accounts, registries should enforce mandatory WebAuthn/FIDO2 hardware keys and strict token binding (OIDC) on the top-100 maintainer cohort identified via multi-objective Pareto optimization ($\alpha = 0.50$). This focused intervention secures over 88% of sole-maintainer risk mass with minimal administrative friction.
- **Differential Release-Authority Gating:** Registries should introduce automated release quarantine

checks that inspect incoming publications for: (a) sudden maintainer roster changes on packages with historical sole maintainers, (b) repository URL mismatches or unanchored `gitHead` commits, and (c) unverified CI build provenance.

- **Supermodular Defense for Critical Ecosystem Assets:** Registries should provide high-assurance dependency verification badges for consumer projects that achieve complete all-source exclusion ($P(S)$) through verified publisher pipelines.

9.2.2 For Downstream Consumers and DevSecOps Organizations

- **Enforce Dependency Cooldown Periods:** Because supply-chain attacks are often detected and yanked within 24 to 72 hours of publication, automated dependency update bots (e.g., Dependabot, Renovate) should enforce a mandatory 3-to-7 day cooldown window for minor and patch updates on non-critical paths.
- **Disable Arbitrary Lifecycle Scripts:** Organizations should configure package managers with `-ignore-scripts` by default, requiring explicit allowlisting for packages that necessitate native compilation during installation.
- **Strict Lockfile Pinning and Integrity Verification:** All CI/CD workflows should enforce immutable lockfile resolution (`npm ci`) with cryptographic subresource integrity checks to prevent dynamic resolution shifts.
- **Prioritize Auditing of Sole-Maintainer Dependencies:** Downstream dependency auditing should prioritize packages where $|M(p)| = 1$, actively sponsoring co-maintainers or contributing upstream code reviews to eliminate single-maintainer bottlenecks.

9.3 Future Work

Future research directions include expanding multi-objective maintainer optimization across multi-language dependency graphs (e.g., Maven, PyPI, and Cargo), incorporating fine-grained dynamic call-graph reachability into exposure formulations, and evaluating the long-term efficacy of our pre-registered prospective cohort protocol through January 2027.

ACKNOWLEDGMENT

The authors thank npm package maintainers and the open-source community for making studied artifacts publicly available, and deps.dev and GitHub for data interfaces. Per venue policy, a large language model was employed solely for non-intellectual grammatical polishing and typesetting assistance; all study design, implementations, analyses, and scientific conclusions were conducted and verified by the authors.

REFERENCES

[1] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel, "Small world with high risks: A study of security threats in the npm ecosystem," in *28th USENIX Security Symposium*. USENIX Association, 2019, pp. 995–1010. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/zimmerman

[2] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," arXiv:2005.09535, 2020, DIMVA 2020 preprint. [Online]. Available: https://arxiv.org/abs/2005.09535

[3] I. Koishybayev, A. Nahapetyan, R. Zachariah, S. Muralee, B. Reaves, A. Kapravelos, and A. Machiry, "Characterizing the security of GitHub CI workflows," in *31st USENIX Security Symposium*. USENIX Association, 2022, pp. 2747–2763. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/koishybayev

[4] T. R. Schorlemmer, K. Okafor, J. Bell, and J. C. Davis, "Signing in four public software package registries: Quantity, quality, and influencing factors," in *45th IEEE Symposium on Security and Privacy (S&P)*, 2024, pp. 1–18.

[5] K. Okafor, S. Torres-Arias, and J. C. Davis, "An industry interview study of software signing for supply chain security," in *34th USENIX Security Symposium (USENIX Security)*. USENIX Association, 2025.

[6] Z. Wang, "Defending the core: A centrality-based protection strategy for supply chain security in npm dependency network," arXiv:2606.14036, 2026, preprint. [Online]. Available: https://arxiv.org/abs/2606.14036

[7] C. Liu, S. Chen, L. Fan, B. Chen, Y. Liu, and X. Peng, "Demystifying the vulnerability propagation and its evolution via dependency trees in the NPM ecosystem," in *44th International Conference on Software Engineering (ICSE)*, 2022. [Online]. Available: https://arxiv.org/abs/2201.03981v2

[8] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, "An analysis of approximations for maximizing submodular set functions—I," *Mathematical Programming*, vol. 14, pp. 265–294, 1978.

[9] N. Zahan, T. Zimmermann, P. Godefroid, B. Murphy, C. Maddila, and L. Williams, "What are weak links in the npm supply chain?" arXiv:2112.10165v2, 2022, ICSE SEIP 2022; DOI: 10.1145/3510457.3513044. [Online]. Available: https://arxiv.org/abs/2112.10165v2

[10] A. Sejfia and M. Schäfer, "Practical automated detection of malicious npm packages," arXiv:2202.13953, 2022, ICSE 2022; DOI: 10.1145/3510003.3510104. [Online]. Available: https://arxiv.org/abs/2202.13953

[11] W. Guo, Z. Chen, Z. Xu *et al.*, "How effective are NPM malicious package detectors? a large-scale empirical study," arXiv:2603.27549v2, 2026, revised August 5, 2026. [Online]. Available: https://arxiv.org/abs/2603.27549v2

[12] S. Muralee, I. Koishybayev, A. Nahapetyan, G. Tystahl, B. Reaves, A. Bianchi, W. Enck, A. Kapravelos, and A. Machiry, "ARGUS: A framework for staged static taint analysis of GitHub workflows and actions," in *32nd USENIX Security Symposium*. USENIX Association, 2023, pp. 6983–7000. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/muralee

[13] P. Weeraddana, M. Alfadel, and S. McIntosh, "Dependency-induced waste in continuous integration: An empirical study of unused dependencies in the npm ecosystem," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 4, pp. 1–28, 2024.

[14] Z. Gao, H. Zhang, L. Chen *et al.*, "Understanding vulnerabilities in software supply chains: An empirical study of function-level reachability," *Empirical Software Engineering*, vol. 30, no. 1, p. Article 20, 2025.

[15] D. Jayasuriya, S. Ou, S. Hegde, V. Terragni, J. Dietrich, and K. Blincoe, "An extended study of syntactic breaking changes in the wild," *Empirical Software Engineering*, vol. 30, no. 2, p. Article 15, 2024.

[16] H. He, B. Vasilescu, and C. Kästner, "Pinning is futile: You need more than local dependency versioning to defend against supply chain attacks," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, p. FSE013, 2025. [Online]. Available: https://arxiv.org/abs/2502.06662

[17] D. Pinckney, F. Cassano, A. Guha, and J. Bell, "npm-follower: A complete dataset tracking the NPM ecosystem," arXiv:2308.12545, 2023. [Online]. Available: https://arxiv.org/abs/2308.12545

[18] I. Rahman, J. Marley, R. Paramitha, and L. Williams, "CHRONO-RESOLUTION: A dependency resolution dataset at release points for npm, PyPI, and crates.io packages," arXiv:2607.15315, 2026, ICSME 2026 Tool Demonstration and Data Showcase Track. [Online]. Available: https://arxiv.org/abs/2607.15315

[19] A. A. Shafin and K. M. Ahmed, "PatchRisk: Forecasting future vulnerability exposure in open-source dependency networks," arXiv:2609.13719, 2026, WI-IAT 2026 preprint. [Online]. Available: https://arxiv.org/abs/2609.13719

[20] P. Ralph, S. Baltes, D. Bianculli *et al.*, "ACM SIGSOFT empirical standards for software engineering research," arXiv:2010.03525, 2020. [Online]. Available: https://arxiv.org/abs/2010.03525

[21] B. Kitchenham, L. Madeyski, and D. Budgen, "Updated guidelines for empirical software engineering research: Avoiding pseudo-replication and fixed-frame fallacies," *IEEE Transactions on Software Engineering*, vol. 51, no. 3, pp. 612–635, 2025.

[22] Open Source Insights, "deps.dev API v3: GetDependencies," Official API documentation, 2026, accessed October 20, 2026. [Online]. Available: https://docs.deps.dev/api/v3/